

M8 — Spring WebServices

Answer Key — Assessment Code M8-A1 (Java / Spring Boot)

SECTION A — Concept Application

SCENARIO 1: You are building a REST API for a food delivery platform using Spring Boot. Your `/menu` endpoint must return a list of available dishes, and a `/menu/{id}` endpoint must return details for a specific dish.

Q1. Explain which annotations you would use on the controller class and methods to handle these two endpoints. Why does `@RestController` behave differently from `@Controller`, and how does `@PathVariable` extract the dish ID from the URL?

The class is annotated `@RestController` and `@RequestMapping("/menu")`. `GET /menu` is handled by a method annotated `@GetMapping`, returning `List<MenuItem>` directly. `GET /menu/{id}` is handled by a method annotated `@GetMapping("/{id}")` whose parameter is annotated `@PathVariable Long id`, which binds the `{id}` segment of the URL to that parameter. `@RestController` is a meta-annotation combining `@Controller` and `@ResponseBody`: `@Controller` alone expects each method to return a view name that a template engine resolves to HTML, whereas `@ResponseBody` tells Spring to serialize the return value straight into the HTTP response body (JSON, via Jackson) instead of resolving a view.

```
@RestController
@RequestMapping("/menu")
public class MenuController {
    private List<MenuItem> menu = new ArrayList<> (List.of(
        new MenuItem(1L, "Paneer Tikka", 220.0),
        new MenuItem(2L, "Veg Biryani", 180.0),
        new MenuItem(3L, "Masala Dosa", 120.0)));

    @GetMapping
    public List<MenuItem> getMenu() { return menu; }

    @GetMapping("/{id}")
    public ResponseEntity<MenuItem> getById(@PathVariable Long id) {
        return menu.stream().filter(m -> m.getId().equals(id)).findFirst()
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }
}
```

Logic:

- `@RestController` = `@Controller` + `@ResponseBody` applied to every method, so every handler's return value becomes the response payload — ideal for a JSON API, not a server-rendered page.
- `@PathVariable` performs URI-template binding: Spring's `HandlerMapping` matches the request path against the method's mapping pattern (`/menu/{id}`) and extracts the `{id}` token, converting it to the parameter's declared type (e.g., `Long`) before the method body runs.
- If the ID does not match any stored `MenuItem`, the method returns `ResponseEntity.notFound().build()` to produce a 404, rather than throwing an unhandled exception.

SCENARIO 2: You are working on the order management module of a food delivery platform. The application must persist customer orders to a MySQL database, and the team wants to avoid writing boilerplate SQL for standard CRUD operations.

Q2. Describe how Spring Data JPA's `CrudRepository` simplifies database interactions compared to raw JDBC. What annotations must an `Order` class carry to be treated as a JPA entity, and what does `@GeneratedValue` contribute to the primary key strategy?

With raw JDBC, every operation (insert, select, update, delete) requires manually writing SQL, opening a Connection, building a PreparedStatement, mapping ResultSet rows to objects, and closing resources. CrudRepository<Order, Long> removes all of that: Spring generates a proxy implementation at runtime that already provides save(), findById(), findAll(), deleteById(), and count() — the developer only declares the interface. For the Order class to be recognized as a persistable entity, it needs @Entity (marks the class as mapped to a table), @Table(name = "orders") (names the table explicitly), @Id (marks the primary-key field), and @GeneratedValue (delegates primary-key generation to the database or JPA provider instead of the application assigning IDs manually).

```
@Entity
@Table(name = "orders")
public class Order {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "customer_name")
    private String customerName;

    @Column(name = "item_name")
    private String itemName;

    private int quantity;

    @Column(name = "total_amount")
    private double totalAmount;
    // getters and setters omitted
}

public interface OrderRepository extends CrudRepository<Order, Long> {
    List<Order> findByCustomerName(String customerName);
}
```

Logic:

- @GeneratedValue(strategy = GenerationType.IDENTITY) tells Hibernate to rely on the database's own auto-increment column, so every new Order automatically receives the next available ID with no manual bookkeeping in Java.
- Because CrudRepository's methods are generated from the interface signature alone, no SQL or implementation class needs to be written for standard operations — Spring Data JPA derives the necessary SQL from the entity's mapping at startup.
- @Column(name = "...") on each field maps a Java field to a specific database column name, which matters when the column naming convention (snake_case) differs from the Java field naming convention (camelCase).

SCENARIO 3: You are securing the admin section of a food delivery backend so that only authenticated users can access /admin/orders. The team also needs incoming order requests to be validated before they reach the service layer.

Q3. Explain how Spring Security's Basic Authentication works and how you would configure it in a Spring Boot application. Additionally, describe how @Valid used alongside @NotBlank and @Min prevents invalid request data from reaching your service methods.

In HTTP Basic Authentication, the client sends an Authorization: Basic <base64(username:password)> header with the request. A Spring Security filter in the security filter chain intercepts every incoming request before it reaches the controller, decodes that header, and validates the credentials against a configured UserDetailsService (in-memory users or a database-backed one). If the header is missing or the credentials are wrong, the filter short-circuits the request and returns HTTP 401 Unauthorized without ever invoking the controller method. @Valid triggers Bean Validation (JSR-380) on the annotated

`@RequestBody` parameter before the controller method body runs; if any field violates a constraint such as `@NotBlank` (must not be null/empty) or `@Min` (must be at least a given value), Spring throws a `MethodArgumentNotValidException`, which is caught by an `@ExceptionHandler` / `@ControllerAdvice` and converted into an HTTP 400 response — the service layer never even sees invalid data.

```
@Configuration
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests(auth -> auth
            .requestMatchers("/admin/**").authenticated()
            .anyRequest().permitAll())
            .httpBasic(Customizer.withDefaults())
            .csrf(csrf -> csrf.disable());
        return http.build();
    }

    @Bean
    public UserDetailsService users() {
        UserDetails admin = User.withDefaultPasswordEncoder()
            .username("admin").password("admin123").roles("ADMIN").build();
        return new InMemoryUserDetailsManager(admin);
    }
}

public class OrderRequest {
    @NotBlank(message = "customerName must not be blank")
    private String customerName;

    @Min(value = 1, message = "quantity must be at least 1")
    private int quantity;
}

@PostMapping("/orders")
public ResponseEntity<Order> place(@Valid @RequestBody OrderRequest request) { ... }
```

Logic:

- A `SecurityFilterChain` bean uses `.requestMatchers("/admin/**").authenticated()` to protect only the admin routes, and `.httpBasic(Customizer.withDefaults())` to enable the Basic Authentication scheme.
- Validation runs at the web layer (before the controller body), so business logic never has to re-check 'is this blank?' or 'is this negative?' — that responsibility is fully delegated to the annotations.
- Because the filter chain runs before routing to any handler, an unauthenticated request to `/admin/orders` never executes any application code — the 401 is returned purely by the security layer.

SCENARIO 4: You are developing a feature in the food delivery platform that calls a third-party logistics partner API to retrieve real-time delivery estimates. The partner API returns a JSON payload containing nested objects and arrays.

Q4. Explain how you would use Java's `HttpClient` to send a GET request to this external API and then use Jackson's `ObjectMapper` to deserialise the JSON response into a Java POJO. What must the POJO's field names match, and how does `@JsonProperty` help when the JSON keys use a naming convention different from standard Java?

An `HttpClient` instance is built once (`HttpClient.newHttpClient()`), then an `HttpRequest` is constructed with the target URI and `.GET()`, and sent synchronously with `client.send(request, BodyHandlers.ofString())`, which returns an `HttpResponse<String>` containing the raw JSON body. Jackson's `ObjectMapper.readValue(jsonString, DeliveryEstimate.class)` then maps that JSON text onto a Java object. By default, Jackson matches each JSON key to a POJO field/getter of the exact same name (case-sensitive), so a JSON key like `estimatedMinutes` maps automatically to a field named `estimatedMinutes`. When the partner API uses a different convention — e.g., snake_case keys like `estimated_minutes`, or an abbreviation the Java side doesn't want to mirror —

`@JsonProperty("estimated_minutes")` is placed on the Java field to explicitly tell Jackson which JSON key maps to that field, decoupling the Java naming convention from the wire format.

```
public class DeliveryEstimate {
    private String orderId;

    @JsonProperty("estimated_minutes")
    private int estimatedMinutes;

    private List<String> route; // maps a JSON array automatically
    // getters/setters
}

HttpClient client = HttpClient.newHttpClient();
HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://partner-api.example.com/estimate/123"))
    .GET().build();
HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());

ObjectMapper mapper = new ObjectMapper();
DeliveryEstimate estimate = mapper.readValue(response.body(), DeliveryEstimate.class);
```

Logic:

- `ObjectMapper` performs the mapping through reflection: it looks for a field or a matching getter/setter pair with the same name as the JSON key by default (case-sensitive, exact match).
- For nested objects and arrays in the JSON payload, the POJO simply declares a nested class (or a `List<T>`) field of the matching structure — Jackson recurses into it automatically.
- `@JsonProperty` is placed directly on the field (or its getter/setter) and overrides the default name-matching rule only for that one field, leaving every other field to match by name as usual.

SCENARIO 5: You are adding an AI-powered meal recommendation feature to the food delivery platform. A `/recommend` endpoint in your Spring Boot application should accept a user's dietary preference and call the OpenAI Chat Completions API to return a personalised suggestion.

Q5. Describe the steps required to integrate the OpenAI API into a Spring Boot REST endpoint. What HTTP method and Content-Type does the Chat Completions API expect, and how would you pass the API key securely without hardcoding it in the source file?

Integration follows four steps: (1) accept the user's preference in a request DTO on POST `/recommend`; (2) build a JSON request body containing "model" and a "messages" array (each message an object with "role" and "content") and send it with Java's `HttpClient` as a POST request to `https://api.openai.com/v1/chat/completions`, setting header `Content-Type: application/json` and `Authorization: Bearer <api-key>`; (3) parse the returned JSON with Jackson's `ObjectMapper` (or `JsonNode` for a quick path lookup) and read `choices[0].message.content`, which holds the assistant's reply text; (4) wrap that text in the endpoint's own response DTO and return it. The Chat Completions API always expects an HTTP POST with `Content-Type: application/json` — never GET, since the prompt payload is sent in the request body. The API key must never sit in a Java source file (it would be committed to version control); instead it is externalized — e.g., `openai.api.key` in `application.properties` sourced from an environment variable (`${OPENAI_API_KEY}`) and injected with `@Value("${openai.api.key}")`.

```
@Value("${openai.api.key}")
private String apiKey;

@PostMapping("/recommend")
public ResponseEntity<Map<String,String>> recommend(@RequestBody Map<String,String> body) {
    String preference = body.get("preference");
    String requestJson = """
    {"model":"gpt-4o-mini","messages":[
    {"role":"user","content":"Suggest a meal for someone who prefers: %s"}]}
    """.formatted(preference);
```

```

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.openai.com/v1/chat/completions"))
    .header("Content-Type", "application/json")
    .header("Authorization", "Bearer " + apiKey)
    .POST(HttpRequest.BodyPublishers.ofString(requestJson))
    .build();
try {
    HttpResponse<String> response = HttpClient.newHttpClient()
        .send(request, HttpResponse.BodyHandlers.ofString());
    JsonNode root = new ObjectMapper().readTree(response.body());
    String reply = root.at("/choices/0/message/content").asText();
    return ResponseEntity.ok(Map.of("recommendation", reply));
} catch (Exception e) {
    return ResponseEntity.status(502).body(Map.of("error", "OpenAI call failed: " + e.getMessage()));
}
}
# application.properties: openai.api.key=${OPENAI_API_KEY}

```

Logic:

- Externalizing the key with @Value plus an environment-variable placeholder keeps the secret out of the compiled JAR's source and out of git history — only the runtime environment holds the real value.
- A JsonNode-based read (mapper.readTree(body).at("/choices/0/message/content").asText()) avoids writing a full POJO hierarchy for a response shape the caller only needs one field from.
- Failures (timeouts, non-2xx status from OpenAI) are caught and converted to a controlled HTTP 502, so a third-party outage never surfaces as an unhandled 500 to the food-delivery app's own clients.

SCENARIO 6: You are building a customer support chatbot for the food delivery platform using LangChain4J. The chatbot must handle recurring questions about order status and delivery times while maintaining a consistent, brand-appropriate tone.

Q6. Explain what a prompt template is in LangChain4J and how it differs from sending a plain string to the OpenAI API. How does chaining multiple prompt steps allow you to build a more context-aware, multi-turn chatbot compared to a single API call?

A PromptTemplate in LangChain4J is a reusable, parameterized instruction string with named placeholders (e.g., "Answer the customer's question about {{topic}} in a friendly tone: {{question}}"). Calling .apply(Map.of("topic", "order status", "question", userInput)) fills in the placeholders and produces a structured Prompt object, which is then sent to the model. This differs from sending a plain concatenated string because the template centralizes tone, format, and instructions in one place — every call to the chatbot reuses exactly the same wrapping instructions, so the brand voice stays consistent no matter what the user typed, instead of that consistency depending on whoever wrote the ad-hoc string that call. Chaining multiple steps — typically via an AiServices interface backed by a ChatMemory — lets the framework automatically prepend prior turns of the conversation (and any retrieved context, such as an order's status from the database) to each new prompt, so the model can refer back to what was said earlier. A single stateless API call has no memory of previous messages; each call is independent, so the user would have to repeat context every time.

```

PromptTemplate template = PromptTemplate.from(
    "You are FoodExpress support. Be brief and friendly.\n" +
    "Order status: {{status}}\nCustomer question: {{question}}");

Prompt prompt = template.apply(Map.of(
    "status", orderService.getStatus(orderId),
    "question", userQuestion));

Response<AiMessage> reply = model.generate(prompt.toUserMessage());

```

Logic:

- A prompt template separates the fixed 'system-style' instructions (persona, tone, constraints) from the variable user input, so the persona cannot be accidentally lost or altered by what the user types.
- ChatMemory (e.g., MessageWindowChatMemory) stores prior turns and is injected into each subsequent prompt automatically, which is what makes a conversation 'multi-turn' rather than a series of unrelated single calls.
- Chaining can also insert a retrieval step (looking up the real order status from the database) between the user's message and the final prompt sent to the model, grounding the chatbot's answer in real data instead of the model guessing.

SECTION B — Practical Coding Tasks

Task 1: Food Menu REST Controller

QB1. Build a Spring Boot REST controller exposing GET /menu, GET /menu/{id}, and POST /menu.

The controller is annotated `@RestController` with a class-level `@RequestMapping("/menu")`. The three endpoints reuse one in-memory `List<MenuItem>` as the data store for the lifetime of the running application.

```
public class MenuItem {
    private Long id; private String name; private double price;
    public MenuItem(Long id, String name, double price) {
        this.id = id; this.name = name; this.price = price;
    }
    // getters and setters
}

@RestController
@RequestMapping("/menu")
public class MenuController {
    private final List<MenuItem> items = new ArrayList<>(List.of(
        new MenuItem(1L, "Paneer Tikka", 220.0),
        new MenuItem(2L, "Veg Biryani", 180.0),
        new MenuItem(3L, "Masala Dosa", 120.0)));

    @GetMapping
    public List<MenuItem> getAll() { return items; }

    @GetMapping("/{id}")
    public ResponseEntity<MenuItem> getById(@PathVariable Long id) {
        return items.stream().filter(m -> m.getId().equals(id)).findFirst()
            .map(ResponseEntity::ok).orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public ResponseEntity<MenuItem> add(@RequestBody MenuItem item) {
        items.add(item);
        return ResponseEntity.status(HttpStatus.CREATED).body(item);
    }
}
```

Logic:

- GET /menu simply returns the whole list; Spring's Jackson message converter serializes it to a JSON array automatically because the class is a `@RestController`.
- GET /menu/{id} streams the list, filters for a matching id, and maps the Optional result to `ResponseEntity.ok(...)` or `ResponseEntity.notFound().build()` — producing 200 or 404 respectively.
- POST /menu reads the incoming JSON into a `MenuItem` via `@RequestBody`, appends it to the list, and returns it wrapped in `ResponseEntity.status(HttpStatus.CREATED)` to produce HTTP 201.

Task 2: Order Persistence with Spring Data JPA

QB2. Build an Order entity, an OrderRepository, and a CommandLineRunner that exercises both.

The Order entity is mapped with `@Entity/@Table/@Id/@GeneratedValue/@Column` as described in Q2. `OrderRepository` extends `CrudRepository<Order, Long>` and adds one derived query method whose implementation Spring Data generates purely from its name at startup — `findByCustomerName` parses to `'SELECT * FROM orders WHERE customer_name = ?'` automatically.

```
@Entity
@Table(name = "orders")
```



```

public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(name = "customer_name") private String customerName;
    @Column(name = "item_name") private String itemName;
    private int quantity;
    @Column(name = "total_amount") private double totalAmount;
    // constructors, getters, setters
}

public interface OrderRepository extends CrudRepository<Order, Long> {
    List<Order> findByCustomerName(String customerName);
}

@Component
public class DataSeeder implements CommandLineRunner {
    private final OrderRepository repo;
    public DataSeeder(OrderRepository repo) { this.repo = repo; }

    @Override
    public void run(String... args) {
        repo.save(new Order(null, "Riya Shah", "Veg Biryani", 2, 360.0));
        repo.save(new Order(null, "Aman Patel", "Paneer Tikka", 1, 220.0));

        System.out.println("All orders: " + repo.findAll());
        System.out.println("Riya's orders: " + repo.findByCustomerName("Riya Shah"));
    }
}

```

Logic:

- Spring Data JPA parses the method name `findByCustomerName` into a query at application startup using its keyword conventions (By, And, OrderBy, etc.) — no `@Query` annotation or SQL string is required.
- The `CommandLineRunner` bean's `run()` method executes once, right after the Spring context has fully started, which is the standard place to seed demonstration data for a console-verified assessment.
- `repository.save(order)` works for both insert and update: because the entity's id is null before saving, Hibernate knows to INSERT and let `@GeneratedValue` assign the primary key.

Task 3: Delivery Status Fetcher using HttpClient and Jackson

QB3. Call <https://jsonplaceholder.typicode.com/todos/1> and print the parsed Todo fields.

A single `HttpClient` instance sends a GET request; the response body (a JSON string) is handed straight to `ObjectMapper.readValue(...)`, which reflectively matches `userId`, `id`, `title`, and `completed` on the `Todo` POJO because those names are identical to the API's JSON keys.

```

public class Todo {
    private int userId, id;
    private String title;
    private boolean completed;
    // getters/setters
}

public class TodoFetcher {
    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newHttpClient();
        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://jsonplaceholder.typicode.com/todos/1"))
            .GET().build();
        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());

        if (response.statusCode() != 200) {
            System.out.println("Request failed: HTTP " + response.statusCode());
            return;
        }
        Todo todo = new ObjectMapper().readValue(response.body(), Todo.class);
    }
}

```



```

System.out.println("User ID : " + todo.getUserId());
System.out.println("Todo ID : " + todo.getId());
System.out.println("Title : " + todo.getTitle());
System.out.println("Completed: " + todo.isCompleted());
}
}

```

Logic:

- No `@JsonProperty` is needed here because the JSON keys (userId, id, title, completed) already match Java field-naming convention exactly — Jackson's default name-based matching is sufficient.
- `response.statusCode()` should be checked before parsing, so a non-200 response produces a clear error instead of a `JsonParseException` on an unexpected body.
- Printing each field individually (rather than `todo.toString()`) satisfies the 'clearly labelled, formatted output' requirement.

Task 4: AI Meal Recommendation Endpoint

QB4. Build POST /recommend, calling OpenAI Chat Completions and returning the suggestion.

The endpoint accepts `{"preference": "..."}]`, builds the Chat Completions request body inline, sends it via `HttpClient` with the two required headers, and extracts `choices[0].message.content` from the response before returning it as JSON.

```

@RestController
public class RecommendController {
    @Value("${openai.api.key}") private String apiKey;
    private final HttpClient client = HttpClient.newHttpClient();
    private final ObjectMapper mapper = new ObjectMapper();

    @PostMapping("/recommend")
    public ResponseEntity<Map<String,String>> recommend(@RequestBody Map<String,String> body) {
        String preference = body.getOrDefault("preference", "balanced");
        try {
            String requestJson = mapper.writeValueAsString(Map.of(
                "model", "gpt-4o-mini",
                "messages", List.of(Map.of("role", "user",
                    "content", "Suggest one meal for a " + preference + " diet."))
            ));
            HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("https://api.openai.com/v1/chat/completions"))
                .header("Content-Type", "application/json")
                .header("Authorization", "Bearer " + apiKey)
                .POST(HttpRequest.BodyPublishers.ofString(requestJson))
                .build();
            HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
            String reply = mapper.readTree(response.body()).at("/choices/0/message/content").asText();
            return ResponseEntity.ok(Map.of("recommendation", reply));
        } catch (Exception e) {
            return ResponseEntity.status(502).body(Map.of("error", "AI service unavailable: " +
                e.getMessage()));
        }
    }
}

```

Logic:

- `Content-Type: application/json` and `Authorization: Bearer <key>` are mandatory headers — omitting either causes OpenAI to reject the request before it is even processed.
- Wrapping the call in `try/catch` and responding with `ResponseEntity.status(502)` on failure meets the requirement to surface a 'Bad Gateway' rather than an unhandled 500 when the third-party API is down.
- Reading the reply with `JsonNode.at("/choices/0/message/content")` is a concise way to reach a single nested field without declaring a full response POJO hierarchy.

SECTION C — Mini Capstone Project

Smart Food Delivery Backend with AI Recommendations

QC1. Expose GET /menu, POST /orders, GET /orders/{id}, and POST /recommend.

This ties Section B's controllers together: MenuController (Task 1) is reused as-is; an OrderController wraps the OrderRepository (Task 2) with POST /orders (validated, persists via repository.save) and GET /orders/{id} (repository.findById); RecommendController (Task 4) is reused unchanged.

```
@RestController
@RequestMapping("/orders")
public class OrderController {
    private final OrderRepository repo;
    public OrderController(OrderRepository repo) { this.repo = repo; }

    @PostMapping
    public ResponseEntity<Order> place(@Valid @RequestBody OrderRequest req) {
        Order order = new Order(null, req.getCustomerName(), req.getItemName(),
            req.getQuantity(), req.getTotalAmount());
        return ResponseEntity.status(HttpStatus.CREATED).body(repo.save(order));
    }

    @GetMapping("/{id}")
    public ResponseEntity<Order> getById(@PathVariable Long id) {
        return repo.findById(id).map(ResponseEntity::ok).orElse(ResponseEntity.notFound().build());
    }
}
```

Logic:

- Delegating persistence to OrderRepository keeps OrderController thin — its job is HTTP concerns (status codes, request/response shape), while JPA/Hibernate handles SQL generation.
- GET /orders/{id} returns 404 via ResponseEntity.notFound().build() when repository.findById(id) comes back empty, exactly like MenuController's GET /menu/{id}.

QC2. Persist orders with Spring Data JPA; configure H2 or MySQL via application.properties.

The Order entity and OrderRepository from Section B, Task 2 are used unchanged. Only application.properties differs between the two datastore choices — the Java code stays identical, which is exactly the abstraction Spring Data JPA is designed to provide.

```
# H2 (quick local run):
spring.datasource.url=jdbc:h2:mem:fooddb
spring.datasource.driverClassName=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true

# MySQL (alternative):
# spring.datasource.url=jdbc:mysql://localhost:3306/fooddb
# spring.datasource.username=root
# spring.datasource.password=yourpassword
# spring.jpa.hibernate.ddl-auto=update
```

Logic:

- spring.jpa.hibernate.ddl-auto=update lets Hibernate create/alter the orders table automatically from the @Entity mapping during development, which is convenient for an in-memory H2 database used only for the assessment run.
- Switching to MySQL in production only means changing the four datasource properties and adding the MySQL driver dependency — no Java code changes.

QC3. POST /recommend must call OpenAI via HttpClient, parse with Jackson, return the suggestion.

Identical in logic to Section B, Task 4 — RecommendController is included in the capstone application as one more @RestController bean, sharing the same api-key property and HttpClient instance pattern described above.

Logic:

- Because Spring manages RecommendController as a bean, apiKey is injected once via @Value at startup, so the same externalized-secret approach from Section A, Q5 applies here without change.

QC4. Protect GET /admin/orders and DELETE /admin/orders/{id} with Basic Authentication.

A SecurityFilterChain restricts /admin/** to authenticated requests only, using .httpBasic(Customizer.withDefaults()). All other endpoints (/menu, /orders, /recommend) remain public via .anyRequest().permitAll(), matching the requirement that only the admin routes need credentials.

```
@RestController
@RequestMapping("/admin/orders")
public class AdminOrderController {
    private final OrderRepository repo;
    public AdminOrderController(OrderRepository repo) { this.repo = repo; }

    @GetMapping
    public List<Order> all() { return (List<Order>) repo.findAll();}

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        repo.deleteById(id);
        return ResponseEntity.noContent().build();
    }
}

@Configuration
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests(auth -> auth
            .requestMatchers("/admin/**").authenticated()
            .anyRequest().permitAll())
        .httpBasic(Customizer.withDefaults())
        .csrf(csrf -> csrf.disable());
        return http.build();
    }
}
```

Logic:

- requestMatchers("/admin/**").authenticated() scopes the restriction narrowly, so /menu and /orders keep working for unauthenticated clients exactly as Section B's tests expect.
- An unauthenticated call to GET /admin/orders never reaches AdminController — the filter chain returns 401 before Spring's DispatcherServlet routes the request to any controller method.

QC5. Validate all incoming request bodies with @Valid, @NotBlank, and @Min; return HTTP 400 on failure.

OrderRequest carries @NotBlank on the string fields and @Min(1) on quantity; the controller method parameter is annotated @Valid. A @RestControllerAdvice class catches MethodArgumentNotValidException globally and converts the list of field errors into one structured JSON body listing every violated field, returned with HTTP 400.

```
public class OrderRequest {
    @NotBlank(message = "customerName must not be blank") private String customerName;
    @NotBlank(message = "itemName must not be blank") private String itemName;
```

```

@Min(value = 1, message = "quantity must be at least 1") private int quantity;
@Min(value = 0, message = "totalAmount cannot be negative") private double totalAmount;
// getters/setters
}

@RestControllerAdvice
public class ValidationExceptionHandler {
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> handle(MethodArgumentNotValidException ex) {
        List<String> errors = ex.getBindingResult().getFieldErrors().stream()
            .map(f -> f.getField() + ": " + f.getDefaultMessage())
            .toList();
        return ResponseEntity.badRequest().body(Map.of("status", 400, "errors", errors));
    }
}

```

Logic:

- Centralizing the exception handler in one `@RestControllerAdvice` means every controller in the capstone (OrderController included) gets the same 400-with-field-list behavior without repeating try/catch logic in each method.
- Because validation happens before the controller body executes, an invalid OrderRequest (e.g., quantity = 0) never reaches `repository.save(...)`, so no bad row is ever written to the orders table.

SECTION D — AI-Augmented Learning

This section is a worked example of the required submission — the exact prompt, the AI tool's original code, the corrected version, and the explanatory note.

1. Exact prompt given to the AI tool

"Write a Java console program that repeatedly reads a prompt from the user, sends it to the OpenAI Chat Completions API using `java.net.http.HttpClient` with an `Authorization: Bearer` header and a JSON body containing model and messages, parses the JSON response with Jackson's `ObjectMapper`, and prints only the assistant's reply text. The loop should continue until the user types 'exit'."

2. AI's original code

```
import java.net.http.*;
import java.net.URI;
import java.util.Scanner;
import com.fasterxml.jackson.databind.*;

public class ChatBot {
    public static void main(String[] args) throws Exception {
        Scanner scanner = new Scanner(System.in);
        String apiKey = "sk-abcdef123456"; // hardcoded key
        ObjectMapper mapper = new ObjectMapper();

        while (true) {
            System.out.print("You: ");
            String userInput = scanner.nextLine();
            if (userInput.equals("exit")) break;

            HttpClient client = HttpClient.newHttpClient(); // recreated every loop
            String body = "{\"model\":\"gpt-4o-mini\", \"messages\": [{\"role\":\"user\", \"content\":\""
                + userInput + "\"}]}";
            HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("https://api.openai.com/v1/chat/completions"))
                .header("Authorization", "Bearer " + apiKey)
                .POST(HttpRequest.BodyPublishers.ofString(body))
                .build();
            HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
            JsonNode root = mapper.readTree(response.body());
            System.out.println("AI: " + root.get("choices").get(0).get("message").get("content").asText());
        }
    }
}
```

3. Corrected version

```
import java.net.http.*;
import java.net.URI;
import java.util.Scanner;
import com.fasterxml.jackson.databind.*;

public class ChatBot {
    public static void main(String[] args) throws Exception {
        Scanner scanner = new Scanner(System.in);
        String apiKey = System.getenv("OPENAI_API_KEY"); // no hardcoded secret
        if (apiKey == null || apiKey.isBlank()) {
            System.out.println("Missing OPENAI_API_KEY environment variable.");
            return;
        }
        ObjectMapper mapper = new ObjectMapper();
        HttpClient client = HttpClient.newHttpClient(); // created once, reused

        while (true) {
            System.out.print("You: ");
            String userInput = scanner.nextLine();
            if (userInput.equalsIgnoreCase("exit")) break; // case-insensitive exit
        }
    }
}
```

```

String body = mapper.writeValueAsString(Map.of(
    "model", "gpt-4o-mini",
    "messages", List.of(Map.of("role", "user", "content", userInput))));

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.openai.com/v1/chat/completions"))
    .header("Content-Type", "application/json") // was missing
    .header("Authorization", "Bearer " + apiKey)
    .POST(HttpRequest.BodyPublishers.ofString(body))
    .build();

HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
if (response.statusCode() != 200) { // status check added
    System.out.println("AI: [request failed, HTTP " + response.statusCode() + "]\n");
    continue;
}
JsonNode root = mapper.readTree(response.body());
System.out.println("AI: " + root.at("/choices/0/message/content").asText());
}
scanner.close();
}
}

```

4. Note explaining the changes

- The AI's version hardcoded the API key directly in source and never set Content-Type: application/json, so OpenAI would reject the request with a 4xx error before ever reaching the model.
- It also recreated a new HttpClient on every loop iteration (wasteful — HttpClient is meant to be reused) and never checked response.statusCode(), so any API error would crash the loop with an unhandled exception when Jackson tried to read a non-success JSON body.
- The corrected version reads the key from an environment variable, builds the client once outside the loop, adds the missing header, checks the status code before parsing, and matches 'exit' case-insensitively so the user isn't stuck if they type 'Exit' or 'EXIT'.